

Lab 2: Text Mining and Chatbot

Ilyes SAIS `ilyes.sais@universite-paris-saclay.fr`

Wenchong PAN `wenchong.pan@universite-paris-saclay.fr`

Quaisar Muhammad `muhammad.qaisar@universite-paris-saclay.fr`

January 22, 2026

1 Lemmatization Approach

In this section, we describe the lemmatization task addressed in Lab 2 and present three different approaches implemented for French lemmatization. The objective is to predict the canonical lemma form of a given surface word, optionally conditioned on its part-of-speech (PoS) tag.

All approaches are trained and evaluated using the datasets provided with the assignment: a modern French training set, a held-out test set, and the historical Gallica dataset used as an out-of-domain evaluation benchmark.

1.1 Problem Definition

Given an input token w and optionally its PoS tag p , the goal of lemmatization is to predict its lemma ℓ . We consider two configurations:

- **Lemma-only**: the output is a character string (e.g., *parlais* $\rightarrow$ *parler*)
- **Lemma + PoS**: the output is implicitly conditioned on the PoS tag

Performance is measured using exact match accuracy between predicted and gold lemmas.

1.2 Dictionary Based Lemmatizer

The first approach is a rule-free dictionary-based lemmatizer. A lookup table is constructed from the training dataset by mapping each observed surface form to its most frequent lemma. When PoS information is used, the dictionary key is extended to $(token, PoS)$, allowing the same surface form to map to different lemmas depending on its syntactic role.

At inference time:

- If the input token (or token PoS pair) exists in the dictionary, the corresponding lemma is returned.
- Otherwise, a default **UNK** symbol is produced.

This method is deterministic, fast, and performs very well when the test data closely matches the training distribution.

1.3 Machine Learning Lemmatizer (Character Level)

The second approach formulates lemmatization as a multi-class classification problem. Each token is represented using character level n -grams extracted with a **CountVectorizer**. A multinomial logistic regression classifier is trained to predict the lemma class.

Two variants are implemented:

- **Without PoS**: only the surface token is encoded
- **With PoS**: the token and its PoS tag are concatenated before vectorization

This approach generalizes better than the dictionary based method, as it can learn morphological regularities (e.g., verb conjugations). However, its performance is limited by the fixed lemma vocabulary observed during training.

1.4 spaCy Lemmatizer

As a reference system, we evaluate the French spaCy lemmatizer (`fr_core_news_sm`). spaCy relies on a combination of rules and statistical models trained on large annotated corpora. It does not require explicit training on the provided dataset and serves as a strong baseline.

1.5 Evaluation Results

Table 1 summarizes the accuracy obtained by each approach on the in domain test set and on the Gallica dataset.

Method	PoS used	Test Accuracy	Gallica Accuracy
Dictionary-based	No	0.94	0.00
Dictionary-based	Yes	0.95	0.00
ML (char n-grams)	No	0.50	0.00
ML (char n-grams)	Yes	0.85	0.00
spaCy baseline	No	0.82	0.00

Table 1: Lemmatization accuracy on test and Gallica datasets.

1.6 Analysis and Discussion

On the in-domain test set, the dictionary-based lemmatizer with PoS information achieves the best performance. This result is expected, as the test data closely follows the same distribution as the training set, allowing exact dictionary matches in most cases.

The machine learning approach benefits significantly from PoS information, confirming that syntactic context reduces lemma ambiguity. However, it remains less accurate than the dictionary-based method due to the large output space and class imbalance.

The spaCy lemmatizer performs competitively without requiring any task-specific training, but it is slightly less accurate than the supervised approaches optimized for this dataset.

A striking observation is that **all methods obtain zero accuracy on the Gallica dataset**. This behavior is not a failure of the algorithms, but a consequence of strong domain mismatch. The Gallica corpus contains:

- historical spelling variants,
- archaic vocabulary,
- obsolete morphological forms,
- Latin tokens and mixed-language content.

Since none of the systems were trained on historical French, most Gallica lemmas are unseen during training and are therefore mapped to UNK, leading to a measured accuracy of zero.

This result highlights the importance of domain adaptation in lemmatization and illustrates the limitations of supervised and dictionary-based approaches when applied to out-of-domain data.

2 BPE algorithm

Byte Pair Encoding (BPE) is a subword segmentation algorithm widely used in modern Natural Language Processing systems, especially for handling rare and unknown words. In this report, we present a guided implementation of the BPE algorithm as proposed by Sennrich et al. (2016), following the steps provided in the assignment.

2.1 Preprocessing

The preprocessing stage prepares the corpus for the BPE training procedure by constructing the core data structures used throughout the algorithm.

First, the raw corpus is segmented into word units using a regular expression. From these tokens, a word-frequency dictionary is built, mapping each distinct word to its number of occurrences in the corpus. This dictionary ensures that frequent words have a stronger influence on the learned merges.

Word frequency representation. Table 2 illustrates the word-frequency dictionary constructed during preprocessing.

Word	Frequency
grecques	42
antique	35
civilisation	18

Table 2: Example of word-frequency dictionary built from the corpus.

The initial vocabulary is defined as the set of all characters appearing in the corpus and is stored as a list in order to allow dynamic extension.

Initial word segmentation. Each word is initially represented as a sequence of characters. This character-level representation is stored in a dictionary called **splits**, as shown in Table 3.

Word	Initial token sequence
grecques	[g, r, e, c, q, u, e, s]
antique	[a, n, t, i, q, u, e]

Table 3: Initial character-level tokenization of words.

At each iteration, the algorithm computes frequencies of adjacent token pairs weighted by word frequency.

Adjacent pair frequency computation. Table 4 shows an example of computed pair frequencies.

Token pair	Weighted frequency
(e, s)	5663
(q, u)	3120
(t, i)	1874

Table 4: Example of adjacent token pair frequencies.

The most frequent pair is selected as the merge candidate at the current iteration.

2.2 BPE Training Procedure

The BPE training procedure iteratively merges the most frequent adjacent token pair until the target vocabulary size is reached.

Merge operation. Once the most frequent pair (a, b) is selected, all occurrences of this pair are merged into a new token ab across the corpus. Conceptually, the merge replaces the two consecutive tokens by their concatenation.

Table 5 illustrates a merge applied to a word representation.

Before merge	After merge
[g, r, e, c, q, u, e, s]	[g, r, e, c, q, u, es]

Table 5: Example of merging the pair (e, s) into the token **es**.

Each merge introduces a new token into the vocabulary and is recorded as a merge rule.

Merge rules accumulation. Table 6 shows an excerpt of the merge rules learned after several iterations.

Merged pair (a, b)	New token
(e, s)	es
(q, u)	qu
(t, i)	ti

Table 6: Example of learned merge rules.

These merge rules are stored in order and later reused to tokenize new input text consistently.

2.3 Detokenization

Detokenization reconstructs readable text from a sequence of BPE tokens. Since tokens are obtained by successive concatenation of characters, detokenization can be performed by concatenating tokens in order.

If word boundaries are explicitly encoded using a special marker, detokenization additionally consists in replacing this marker with whitespace. Table 7 illustrates a simple detokenization example.

Token sequence	Detokenized text
[g, r, e, c, q, u, es]	grecques

Table 7: Example of detokenization from BPE tokens to text.

3 Discussion and Conclusion

This report addressed two fundamental problems in Natural Language Processing: lemmatization and subword tokenization. Through the lemmatization task, we explored how different modeling choices (dictionary-based, machine learning, and rule-based systems) impact performance depending on the availability of linguistic information and the target domain. Our experiments show that simple dictionary-based approaches remain highly effective when training and test data are drawn from the same distribution, especially when PoS information is available. In contrast, machine learning approaches provide better generalization capabilities but remain constrained by the fixed output vocabulary observed during training. The evaluation on the Gallica dataset highlights the critical role of domain adaptation: historical spelling, archaic morphology, and mixed-language content strongly limit the applicability of models trained exclusively on modern French.

In the second part, we implemented the Byte Pair Encoding algorithm to study subword segmentation from first principles. This experiment illustrates how frequency-driven merges allow models to construct meaningful subword units and efficiently handle rare or unknown words. The BPE algorithm provides a robust compromise between character-level and word-level representations and explains why subword tokenization has become a standard component of modern neural language models.

Overall, these two labs emphasize the importance of representation choices in NLP pipelines. While linguistic resources such as dictionaries and PoS tags can significantly improve performance in controlled settings, robust language processing in realistic scenarios requires careful consideration of domain variation and data distribution.